

DISEÑO DE UNA SOLUCIÓN DE INTEGRACIÓN EN EL ÁMBITO DE UNA FÁBRICA

Claudia Moreno Martínez
Finn Alai Perea Oltmann
Ainhoa López Gómez
Paula Barona Terol

PR2-2-6



ÍNDICE

1. Introducción y Objetivos.....	2
2. Definición del Proyecto Automatizado.....	7
3. Escenarios de Integración y Arquitectura de Comunicaciones.....	9
4. Desarrollo del Software de Control y Comunicación.....	12
5. Integración con otras asignaturas.....	16
5.1. Descripción del Sistema de Información.....	17
5.2. Diagrama UML.....	18
5.3. Esquema Lógico - Relacional.....	21
6. Calidad y Adecuación del Proyecto.....	24

1. Introducción y Objetivos

La industria española de cerámica representa el 22.2% del PIB industrial de la Comunidad Valenciana y supone más del 32.2% del total del PIB de la provincia de Castellón. En este contexto, el control de calidad, empaquetado y paletizado de azulejos constituye una de las áreas con mayor impacto tanto en la productividad como en la calidad final del producto. Tradicionalmente, estas áreas se realizaban de manera manual apoyándose en la experiencia y visionado del operario; además de la capacidad física para transportar pesos durante una larga jornada laboral. Sin embargo, este enfoque presenta diversas limitaciones, como la fatiga visual o muscular de los trabajadores, riesgo de lesiones y variabilidad tanto en el rendimiento como en la eficacia del proceso.

El proyecto desarrollado propone la automatización integral de este proceso mediante la integración de diferentes tecnologías. Para evaluar la eficiencia de la propuesta, se han definido una serie de indicadores claves de rendimiento (KPIs) que permiten medir de forma objetiva el impacto de la automatización.

Para la definición de los KPIs asociados a cada criterio, a continuación se va a comentar cuales han sido los elegidos y la relación entre las métricas, técnicas y beneficios obtenidos. Previamente, se va a definir el concepto de KPI para una mayor determinación de estos.

Las siglas KPI (*Key Performance Indicator*) es el término que se utiliza en el ámbito laboral como indicador clave de rendimiento. Estos indicadores son las medidas o números que se utilizan para medir el desarrollo de una compañía. De esta manera, las empresas pueden medir tanto el grado de satisfacción que tienen sus clientes, como la facturación anual del porcentaje de ventas de los servicios o productos generados.

En primer lugar, el éxito de objetivos de productividad y rendimiento del proyecto, viene determinado por la capacidad del sistema automatizado para igualar o superar el ritmo de producción de la planta actual (trabajo manual) sin generar cuellos de botella. Para ello el parámetro a estudiar (KPI de productividad) ha sido el tiempo de ciclo. Se ha evaluado el tiempo total desde que un azulejo es detectado por la cámara Matrix Iris GTR hasta que la caja llena es posicionada por el UR30 en el palé de 800x800 mm. Para una mayor justificación a continuación se va a comentar el retorno de inversión o ROI, del proyecto.

Para obtener el retorno de la inversión se ha elaborado un documento Excel que recoge lo siguiente:

PRESUPUESTO DEL PROYECTO			PRECIO	CANTIDAD
UR30			51.495,00 €	1
UR5e			27.557,00 €	1
Garra SMC ZWP7A01-ZP2OU-X1 Vacuum Gripper			2.110,26 €	1
Sensor SICK WL4S			189,55 €	5
Garra OnRobot VGP20 Vacuum gripper			7.657,81 €	1
Cámara Matrox Iris GTR			2.500,00 €	1
Cinta mecánica			4.906,55 €	5
Palé			13,51 €	2
TOTAL COSTES			96.429,68 €	116.827,59 €

Lo primero para poder trabajar en el retorno de inversión se debe calcular aproximadamente la cifra total a la que asciende el proyecto. Para ello, se ha dividido en diferentes secciones; la primera que se puede observar es la tabla del presupuesto del proyecto. En esta tabla se tiene en cuenta todos los elementos que conforman la celda. Para obtener el valor total del presupuesto se han utilizado precios aproximados del mercado de cada uno de los elementos. Como se puede observar hay dos celdas que corresponden al total de costes; la celda de la columna de precio es la suma total de los elementos; la celda situada más a la derecha es la que corresponde a la suma total de todos los elementos de la celda. Esto es así porque hay elementos como las cintas mecánicas que requieren de 5 unidades, al igual que en el caso de los sensores.

MANTENIMIENTO ANUAL			
Mantenimiento del UR30		1.500,00 €	
Mantenimiento del UR5e		1.200,00 €	
Recambio de las ventosas		500,00 €	
Recambio de las correas de las cintas		270,00 €	
Consumo eléctrico		2.500,00 €	16 h/día
TOTAL COSTES			5.970,00 €

De igual manera, para poder obtener la cifra total del proyecto también se debe tomar en cuenta el mantenimiento anual que conllevan las unidades robóticas, junto a las ventosas de ambas herramientas de trabajo de los cobots y el consumo eléctrico que se ha calculado que asciende a un total de 16 horas al día. Esto se ha calculado así debido a que se ha supuesto que en la fábrica existen 2 turnos de trabajo, uno de mañanas y otro de tardes, con una jornada laboral de 8 horas.

INSTALACIÓN			
Ingeniería y programación (Robo DK)		80,00 €	200 hora/s
Montaje mecánico y cableado		8.500,00 €	
Cuadro eléctrico y seguridad		4.500,00 €	
Transporte y puesta en marcha		2.500,00 €	
Instalación del UR30		8.500,00 €	
Instalación del UR5e		10.500,00 €	
TOTAL COSTES			50.500,00 €

Para continuar, también se ha tenido en cuenta el coste de la instalación. Esta tabla se divide en diferentes categorías; ingeniería y programación, montaje mecánico y cableado, cuadro eléctrico y seguridad, transporte y puesta en marcha y la instalación del UR30 y del UR5e. En cuanto al apartado de ingeniería y programación se ha tenido en cuenta solamente las horas de ingeniero. El ingeniero es el encargado de escribir y depurar el código, de configurar la cámara y de programar ambas unidades robóticas. Se ha aproximado que un proyecto de estas medidas implicaría alrededor de 150-200 horas de ingeniería, por lo que habría que multiplicarlo por el salario por hora de un ingeniero. Esto último ronda entre los 60-80€.

Seguidamente, la parte de montaje mecánico y cableado equivale a la parte física del proyecto. En esta sección es imprescindible no tener en cuenta los anclajes de las bases de los robots UR30 y UR5e; además, alinear y montar las 5 cintas transportadoras con sus respectivos sensores. Y, por último, la gestión del cableado.

Esto enlaza con la siguiente sección, el cuadro eléctrico y la seguridad. Aunque no se necesiten medidas de seguridad como vallado/jaulas sí se requiere de la instalación de controladores o también conocidos como PLCs, materiales como relés o fuentes de alimentación y cableados y medidas de seguridad para el cumplimiento de las normativas vigentes.

Luego, una vez está todo diseñado, los ingenieros como el equipo técnico encargado del montaje deben estar un par de días o incluso 1 o 2 semanas en la fábrica llevando a cabo la fase de prueba del proyecto. Esta fase es de suma importancia porque es ahí donde se ajustan los sensores que fallan, se modifican los alcances, se calibran posiciones y trayectorias de los robots para garantizar un correcto funcionamiento en la celda.

Para finalizar con la tabla, cabe justificar los motivos por los que la instalación del UR5e es más cara que la instalación del UR30. Esto es debido a que el UR5e requiere de coordinarse él mismo con tres cintas mecánicas y una cámara. Asimismo, su parte del proceso (control de calidad y empaquetado) obliga a implementar un mayor grado de precisión en comparación del trabajo que realiza el UR30.

Todos estos aspectos, en conjunto hacen que la suma total del presupuesto del proyecto alcance la cifra aproximada de 175.000 € .

TOTAL PRESUPUESTO DEL PROYECTO	173.297,59 €
---------------------------------------	---------------------

Una vez se ha obtenido ya la cifra total de la inversión se puede abarcar ya el apartado de recuperación de la inversión.

RECUPERACIÓN DE LA INVERSIÓN						
Concepto	AÑO 0	AÑO 1	AÑO 2	AÑO 3	AÑO 4	AÑO 5
Inversión: Coste del proyecto	- 173.297,59 €	- €	- €	- €	- €	- €
Ahorros: salarios	- €	100.000,00 €	100.000,00 €	100.000,00 €	100.000,00 €	100.000,00 €
Gastos: Mantenimiento	- €	- 5.970,00 €	- 5.970,00 €	- 5.970,00 €	- 5.970,00 €	- 5.970,00 €
FLUJO ANUAL	- 173.297,59 €	94.030,00 €	94.030,00 €	94.030,00 €	94.030,00 €	94.030,00 €
TOTAL ACUMULADO	- 173.297,59 €	- 79.267,59 €	14.762,41 €	108.792,41 €	202.822,41 €	296.852,41 €

El año 0 no se contempla ni gastos en mantenimiento ni ahorro en salarios. Sin embargo, a partir del año 1 sí se contemplan. Para una mayor comprensión de los datos a continuación se va a explicar el origen de cada apartado; inversión, ahorros en salarios y gastos en mantenimiento.

El valor tanto del apartado de inversión como el valor del apartado en gastos, equivalente al coste del proyecto y al mantenimiento, son las cifras calculadas anteriormente. Sin embargo, el valor de 100.000€ en ahorros en salarios anuales surge de calcular el coste laboral total de los operarios que dejarán de realizar tareas manuales repetitivas para pasar a otras funciones. Para el cálculo se ha considerado la existencia de 2 turnos diarios (mañana y tarde) con una jornada laboral de 8 horas. Asimismo, se estiman 220 días laborables al año. A continuación, para el control de calidad y empaquetado se requiere a un operario por turno que se dedique a inspeccionar visualmente los azulejos y depositarlos en los interiores de las cajas correspondientes. Mientras que en el puesto final de la línea de trabajo, el paletizado de las cajas requiere mínimo el trabajo de un operario que sea el encargado de cargar exclusivamente las cajas llenas de azulejos y que las deposite en la posición correspondiente del palé. En total se ahorrarían mínimo 2 operarios por turno, es decir, un total de 4 operarios. Para el cálculo se ha considerado que el sueldo anual bruto de un operario alcanza los 25.000€.

$$\text{Total ahorro en salarios: } 4 * 25.000 \text{ €} = 100.000 \text{ €}$$

Por último, para obtener más información de la inversión del proyecto hay que calcular el *PayBack* del proyecto y el ROI.

PayBack	1,84300319 años
ROI	171,30%

PayBack muestra que la inversión inicial se recuperará totalmente en 1 año y 10 meses aproximadamente. En la industria, el umbral de riesgo suele situarse en los 3 años. estar por debajo de los 2 años, como es el caso de este proyecto, lo coloca en la categoría de inversiones de alta prioridad. En este caso, a partir de los 2 años la empresa comienza a tener un flujo de caja positivo, lo cual le permitirá abordar otras necesidades.

En suma, el ROI señala que el beneficio casi triplica el coste inicial. Financieramente, este porcentaje tan alto ofrece un colchón muy amplio a la empresa que decida implementar el proyecto. Incluso si surgieran imprevistos como paradas de producción o roturas en las unidades robóticas, el proyecto seguiría siendo rentable, ya que el margen de beneficio es muy superior al coste.

La segunda categoría a comentar es la calidad de clasificación y precisión de posicionado del azulejo. Se busca garantizar que la automatización no solo sea más rápida, sino también más exacta que la calidad humana. Es por ello que el parámetro a estudiar (KPI)

ha sido la eficacia o también conocida como la tasa de error. Es decir, la lógica de control debe demostrar una tasa de éxito de al menos el 98% en la separación de piezas según su estado (correcto, defectuoso o roto), acorde a los estándares de la industria de la cerámica. Esto es posible gracias al uso de la cámara de visión artificial implementada en la celda. Este elemento permite identificar y clasificar detalles mínimos como el color o el brillo con una precisión mayor y duradera a lo largo de la jornada laboral completa. Seguidamente, se ha analizado también el parámetro de la precisión, de manera que el sistema aproveche la repetibilidad de las unidades robóticas colaborativas, UR30 y UR5e, para asegurar que los azulejos se depositen sin colisiones laterales dentro de la caja. La automatización del proceso va a reducir la tasa de error gracias a la diferencia de capacidad de constancia y precisión entre un proceso automatizado y un trabajador. Todos estos aspectos en conjunto permiten a la empresa reducir el porcentaje de errores y, por ende, no tener tantos desperdicios económicos.

Para el estudio de la flexibilidad y la escalabilidad, se ha analizado y mejorado el parámetro (KPI) del tiempo de reconfiguración. Este parámetro mide el tiempo necesario para adaptar la celda a un nuevo formato ya sea de azulejo o una variante en las distancias entre los objetos del espacio de trabajo. Así pues, en caso de ser necesaria una actualización, la organización de programas por carpetas, y la estructura robusta de la celda en frames, permite hacer las modificaciones necesarias sin tener que reescribir la lógica de todos los programas o reorganizar los objetos. Por ello, se considera un éxito que al modificar las dimensiones de la pieza el proceso se acopla automáticamente sin errores.

Y para finalizar, en cuanto a la categoría de seguridad del operario en el espacio de trabajo, se ha pretendido eliminar las exigencias físicas del entorno, así como la reducción de riesgos. Se busca validar que el hardware seleccionado pueda soportar el trabajo que previamente llevaban a cabo los operarios. Para el éxito de éste, se debe tener en cuenta dos parámetros; el parámetro de la gestión de carga de las unidades robóticas, más específicamente del UR30, puesto que es el cobot que se hace cargo del paletizado de las cajas de azulejos. Y el parámetro del alcance del robot en el espacio de trabajo. De este modo, se ha asegurado que no existan puntos con singularidades, es decir, puntos donde el robot se bloquee y no pueda llevar a cabo la tarea que está realizando y, además que alcance en este caso todas las esquinas del palé.

2. Definición del Proyecto Automatizado

Este proyecto se centra en el control de calidad, empaquetado y paletizado del producto. De este modo, la automatización de dicho proceso se divide en dos partes explicadas a continuación.

En primer lugar, la primera parte del proceso cubre el control de calidad y empaquetado de los azulejos. Hasta el momento, esta parte se realizaba de forma manual mediante observación y empaquetado hechos por el trabajador. Este proyecto propone sustituir la mano de obra por una cámara encargada de determinar el estado del azulejo (buen estado, defectuoso o mal estado), y un brazo robótico encargado de empaquetar los azulejos según corresponda. Dicha distinción es importante, ya que los azulejos se depositarán en diferentes cajas dependiendo de sus condiciones. Para ello, los azulejos pasarán por una cinta transportadora mientras son analizados por la cámara, que estará colocada sobre la cinta a una altura adecuada.

No obstante, antes de continuar con el proceso cabe comentar que debido a las circunstancias del proyecto, no es posible la programación e implementación de este elemento (Cámara de Visión Artificial) para su demostración, por lo que será sustituido por un sistema embebido (ESP32-S3). De esta manera, el operario contará con 3 botones -uno por cada estado posible del azulejo- y una vez pulsado, esta información se enviará a través de un broker de comunicación (MQTT) a la aplicación Python, quién mandará ejecutar las tareas correspondientes a las unidades robóticas.

Seguidamente, cuando los azulejos estén situados en su zona de pick (final de la cinta transportadora), el brazo se dispondrá a empaquetar el producto.

Así pues, existen dos “tipos” de cajas. En el caso en el que los azulejos estén en buen estado el brazo robótico se encargará de agarrarlo y depositarlo dentro de la caja de empaquetado de azulejos en buen estado con la mayor precisión y delicadeza hasta llenarla. Asimismo, si se trata de un azulejo con algún defecto, ya sea en su forma o en su diseño, el robot repetirá el mismo proceso, pero esta vez depositará dicho azulejo en una caja diferente la cuál también se pondrá a la venta pero con un precio más reducido, ya que se trata de azulejos defectuosos. Sin embargo, si se trata de un azulejo roto o con alguna deformidad más notoria, éste se depositará en la zona de reciclaje, es decir, se dejará caer conforme avanza la cinta hasta dicho espacio, puesto que no es un material que se pueda reutilizar.

En segundo lugar, una vez las cajas están listas, se desplazarán mediante otra cinta transportadora al segundo área de trabajo. De este modo, comienza la segunda parte del proceso, es decir, el paletizado: una vez estén las cajas situadas en su zona de pick - habiendo dos zonas correspondientes a los dos tipos de cajas - un segundo brazo robótico se encargará de llenar los palets asignados con las cajas listas y depositarlas de forma correcta.

Una vez finalizado el proceso, el palet queda listo para su retirada y posterior venta. Sin embargo, cabe mencionar que se da por supuesto la retirada de los palets llenos y su reposición adecuada, ya que este proyecto se centra exclusivamente en las tareas mencionadas anteriormente, es decir, el control de calidad, empaquetado y paletizado de los azulejos.

En cuanto a la implementación con la ESP32-S3, esta se ha programado utilizando el entorno de Arduino para gestionar tanto las entradas como las salidas del sistema, así como la comunicación con el resto de la automatización. Por una parte, la ESP32-S3 se encarga de leer distintos botones que simulan acciones del proceso, como el tipo de azulejo (según su estado) y el vaciado de palets. Estas señales se envían mediante el protocolo MQTT, utilizando distintos *topics*, lo que permite integrarlo dentro de una arquitectura cliente-servidor y comunicarse con otros dispositivos del sistema.

Por otra parte, la ESP32-S3 también recibe información a través de MQTT, la cual se utiliza para controlar varios LEDs. Estos LEDs indican diferentes estados del proceso, como la activación del proceso o si un palet está lleno. De esta forma, se consigue una comunicación bidireccional, donde la ESP32 no solo envía información, sino que también recibe datos en tiempo real, facilitando la supervisión del sistema.

Respecto a la BD, esta constituye el núcleo de información del proyecto, cuya función principal es centralizar y procesar de forma continua los datos generados tanto por el sistema embebido ESP32-S3 como por las unidades robóticas UR5e y UR30. A través de una conexión directa con la aplicación en Python, la base de datos actúa como un registro dinámico que captura cada evento del proceso de clasificación y paletizado en tiempo real.

Cada vez que el sistema identifica un azulejo, se registra su estado de calidad, lo que permite alimentar un histórico detallado de la producción y detectar patrones de fallos. Asimismo, cuando se completa una caja o se llena un palet, la base de datos actualiza automáticamente los niveles de inventario y el estado de las zonas de carga. Esta actualización constante es la que permite el cálculo automatizado de los KPIs críticos, como la tasa de errores de clasificación, la eficiencia del tiempo de ciclo y la salud del inventario.

En última instancia, la base de datos transforma las señales de hardware en información estratégica. Esto no solo facilita la supervisión inmediata del estado general del almacén, sino que proporciona a los responsables de planta una herramienta de trazabilidad total, permitiendo tomar decisiones basadas en datos reales para optimizar el rendimiento y reducir las mermas en la cadena de producción.

Para finalizar, la automatización del proceso de control de calidad, empaquetado y paletizado de azulejos permitiría a la empresa una reducción en costes y desperdicios determinante para la economía de esta. Clasificar correctamente los azulejos permite recuperar material para la venta de segunda categoría, maximizando el aprovechamiento del producto terminado.

3. Escenarios de Integración y Arquitectura de Comunicaciones

Dentro del proyecto de automatización, se definen diferentes escenarios de integración que permiten coordinar el funcionamiento entre los distintos sistemas implicados. En cada uno de estos escenarios intervienen dispositivos como la ESP32-S3, las unidades robóticas y aplicaciones externas desarrolladas en lenguajes como C++ o Python, que en conjunto permiten el control y la supervisión del proceso completo.

La ESP32-S3 actúa como el elemento central de comunicación del proyecto, ya que es la encargada de leer las entradas físicas del sistema y enviar información de vital relevancia sobre el proceso a los operadores.

Así pues, existen dos escenarios de integración; el primer escenario consiste en la gestión de entradas y clasificación de material, y el segundo escenario de integración se basa en la gestión logística de paletizado y comunicación con los operarios.

Respecto al primer escenario de integración se deben tener en cuenta diferentes participantes; el dispositivo embebido ESP32-S3, la aplicación en Python y las unidades robóticas UR5e y UR30.

La ESP32-S3 actúa como un sensor, cuya tarea principal es la monitorización de 5 entradas digitales a través de sus pines GPIO. En total, el sistema embebido gestionará las posibles pulsaciones de 5 botones; tres de los botones serán implementados para la clasificación según el estado del azulejo (roto, bueno, defectuoso), simulando el funcionamiento determinista de lo que en realidad sería una cámara inteligente. El resto de botones hacen referencia a los palets, es decir, cuando se requiera de la retirada de algún palet, el operario pulsará el botón correspondiente a ese palet -haciendo distinción entre dos zonas de trabajo diferentes-. De igual manera que en los azulejos, cada palet cuenta con un botón independiente en base al estado de los azulejos (defectuosos o bueno).

Si se trata de publicar la situación dónde un palet se encuentra listo para su retirada y posterior distribución, se llevará a cabo mediante el *topic* `"sim/working/button/empty"`. Ahora bien, si se trata de la clasificación de los azulejos en base a la calidad de su estado, éste no se realizará por el mismo canal, sino que se realizará a través del *topic* `"sim/working/button/spawn"`. Se ha decidido separar ambos casos en canales diferentes para evitar posibles interferencias; además, esta estructura de canales permite una mayor organización a la hora de ejecutar el proceso de comunicación.

El encargado de publicar en la red es el broker MQTT, cuyo rol se puede definir como middleware de mensajería. MQTT actúa garantizando que el controlador reciba la información de manera asíncrona y gestione la comunicación entre todos los participantes del proceso.

En el caso del controlador, éste haría referencia a la aplicación Python, el cual es el encargado de la inteligencia del sistema. Está suscrito a los *sub-topics* de *button* y tras recibir un mensaje u otro en el topic requerido, ejecuta las instrucciones necesarias para que la unidad robótica correspondiente siga la trayectoria o realice la tarea correctamente. Es por ello que tanto la unidad robótica UR5e como la unidad robótica UR30 funcionan como actuadores. Sin embargo, si la información recibida hace referencia a la retirada del palet, ésta no se transmitirá a ninguna API en Python, sino que se enviará a los operarios de la planta para que puedan llevar a cabo la retirada del palet que se encuentre completo.

Respecto a la programación en la simulación del entorno RoboDK se ha optado por eliminar los elementos que conforman los palets una vez se haya obtenido el pulso del botón correspondiente. No se ha llevado a cabo la simulación de la retirada porque se considera un proceso externo al proceso que se busca automatizar en el proyecto.

De este modo, se puede afirmar que en el primer escenario de integración se implementa el sistema embebido como emisor -con sus respectivas entradas GPIO provenientes de los botones- y el script controlador (API en Python) actuará como receptor de esta información recibida.

Por otra parte, en cuanto al diseño de la comunicación, todo el sistema se basa en el protocolo MQTT, siguiendo una arquitectura cliente-servidor basada en el modelo publicador-suscriptor. En general, la ESP32-S3 actúa principalmente como dispositivo publicador, enviando información relacionada con los eventos del sistema, aunque posteriormente, ésta cambiará de rol y pasará a tener una función de actuador, ya que a su vez también gestiona las salidas mediante indicadores LED que muestran el estado del proceso. No obstante, eso se explicará más detalladamente en el segundo escenario de integración.

Finalmente, el segundo escenario de integración complementa la arquitectura al tratarse de la parte del proceso donde la ESP32-S3 comunica el estado operativo de la planta al operario. En esta fase del proyecto los participantes son los mismos pero con ligeras modificaciones en sus roles respecto al anterior escenario de integración; unidades robóticas UR5e y UR30, aplicación Python, broker MQTT y el sistema embebido ESP32-S3.

En esta fase del proyecto, las unidades robóticas UR5e y UR30 son las encargadas de generar eventualidades, como por ejemplo; el proceso está activo o el palet está listo para su retirada. A través de la API de RoboDK, vinculada al controlador lógico (aplicación Python), la unidad robótica correspondiente, en este caso el cobot UR30, informa sobre la eventualidad del proceso. Para que de esta manera, la aplicación Python procese la señal que ha enviado la unidad robótica, interprete los datos recibidos y publique el estado -proceso activo, palet 1 o palet 2 lleno- a través del broker.

En cuanto al broker MQTT, éste no ha sufrido ninguna alteración en su rol. Al revés, la función del MQTT sigue siendo la misma, función de intermediario de manera bidireccional. Sin embargo, la ESP32-S3 en este caso asume el rol de suscriptor. Su tarea principal es la escucha activa de los *topics* de estado para controlar todos los elementos

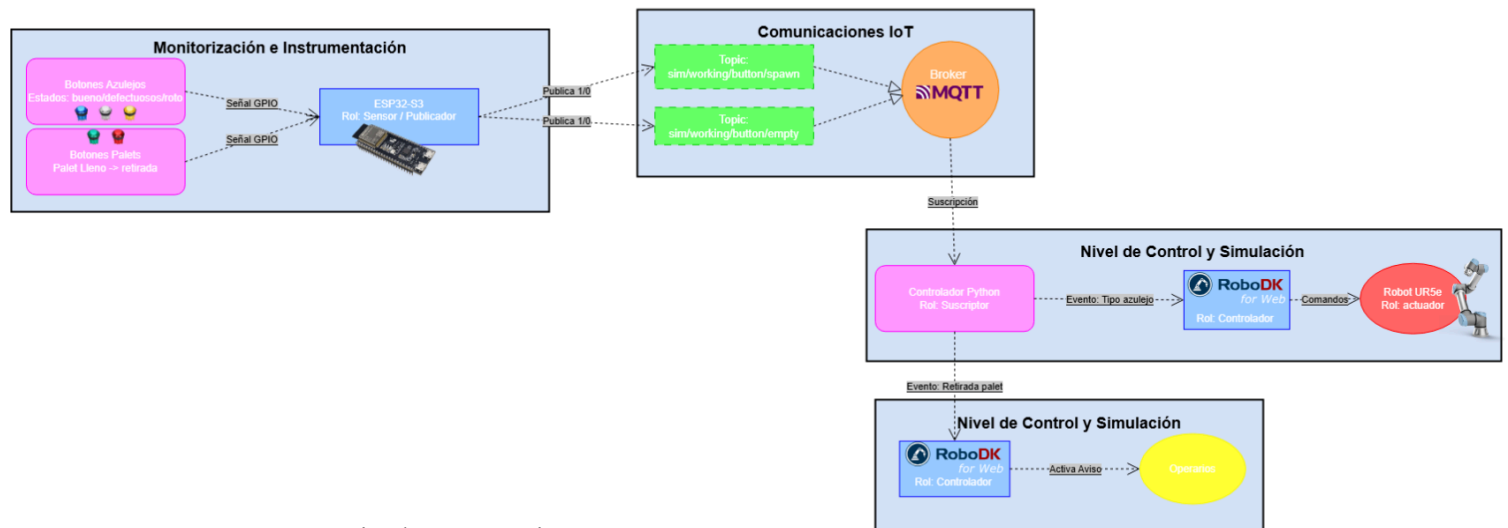
indicadores de la planta de trabajo. Para indicar que un proceso ha llegado a su final o que está activo se va a hacer uso de indicadores LEDs. Además, cabe destacar que al tratarse de un flujo de información diferente al anterior, se hace uso de un canal diferente. De manera que para informar de eventualidades del proceso de automatización se hará uso de los siguientes *topics*: "*sim/working/palet1*" y "*sim/working/palet2*" para indicar que el palet1 o el palet2 están llenos de manera respectiva. Por último, para la realización del aviso del proceso activo se hará uso del *topic* "*sim/working*", el cual enviará una señal de encendido si el proceso arranca sin errores, y de apagado en caso de que finalice el proceso.

Continuando con la zona de palets, cuando un palet se encuentre al máximo de su capacidad, a parte de un indicador LED que actuará como aviso visual a los operarios de la planta, también se paralizará esa zona del proceso hasta que el palet no sea retirado. De esta manera, se evitará posibles situaciones de saturación de trabajo.

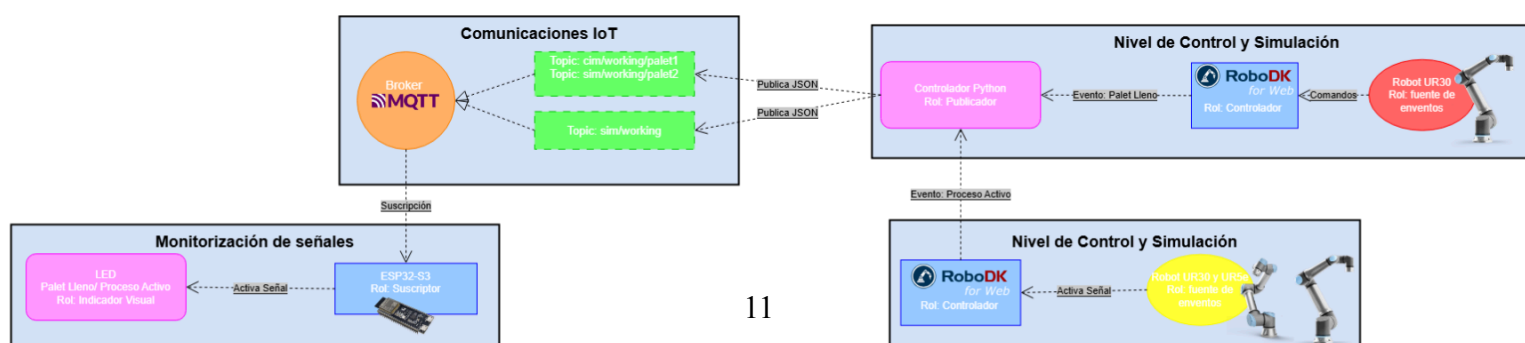
Finalmente, para que ambos flujos de información funcionen correctamente y evitar conflictos en la comunicación, ya sea para el primer o segundo escenario, la información enviada serán tipo de datos numéricos -que indicarán de qué tipo de azulejo se trata, si se debe retirar el palet- o los comandos "on" u "off", todos ellos en sus topics correspondientes.

En síntesis, con el fin de una mayor claridad en cuanto los participantes de cada escenario de integración y los diferentes roles que toman, se ha optado por la elaboración de un esquema visual que clarifique todos estos aspectos.

Escenario de Integración 1:



Escenario de Integración 2:



4. Desarrollo del Software de Control y Comunicación

En cuanto al diseño de la arquitectura de Comunicaciones, específicamente la programación y manejo de la ESP32-S3, se ha utilizado como base el *template* proporcionado por la asignatura *Proyecto Rii 2: Desarrollo y Programación*. De este modo, con la edición y adaptación al proyecto concreto, el código puede dividirse en 9 archivos fundamentales para el funcionamiento de comunicaciones.

En primer lugar, para las configuraciones necesarias en cuanto a conexiones y manejo de datos, son necesarios los siguientes cuatro archivos. El primero, se trata del archivo de configuración “*Config.h*”, el cual se encarga de definir todos los parámetros generales del sistema, incluyendo la velocidad de comunicación serie, la configuración de la red WiFi, los datos del servidor MQTT y la asignación de pines de entrada y salida del ESP32. Además, en este archivo se establecen los diferentes *topics* MQTT empleados para la comunicación entre el dispositivo físico y la simulación, permitiendo gestionar tanto el estado de funcionamiento como el control de los palets y los botones de generación de azulejos.

El segundo archivo, “*logger.ino*”, implementa un sistema de registro y depuración mediante consola serie. Este módulo permite mostrar mensajes clasificados según distintos niveles de severidad, como *TRACE*, *DEBUG*, *INFO*, *WARN*, *ERROR* y *FATAL*. Gracias a este sistema, resulta más sencillo monitorizar el comportamiento del dispositivo y detectar posibles errores durante la ejecución del programa o durante la comunicación con el broker MQTT y la red WiFi.

Por otra parte, el archivo “*wifi_lib.ino*” se encarga de gestionar la conexión inalámbrica del ESP32 a la red WiFi definida previamente en el archivo de configuración. Este módulo verifica continuamente el estado de la conexión y permite mantener estable la comunicación con la infraestructura de red, garantizando así que el dispositivo pueda intercambiar información con el broker MQTT y el resto de componentes del sistema.

Finalmente, el archivo “*mqtt_lib.ino*” implementa toda la lógica relacionada con la comunicación MQTT. Entre sus funciones principales se encuentran la conexión y reconexión automática al broker, la suscripción a los distintos *topics*, la publicación de mensajes y la recepción de datos mediante funciones *callback*. De esta manera, el sistema puede enviar información sobre el estado de los palets y recibir órdenes externas, permitiendo la sincronización entre el entorno físico y la simulación desarrollada.

En segundo lugar, se implementan los archivos encargados de la configuración física del dispositivo y de las funciones auxiliares necesarias para el control del sistema. El archivo “*funciones.ino*” contiene funciones de apoyo utilizadas durante la ejecución del programa, destacando especialmente la función *setInternalLed()*, encargada de controlar el LED integrado de la placa ESP32-S3. Esta función permite cambiar el estado del LED únicamente

cuando existe una variación real en su estado, evitando escrituras innecesarias sobre el pin digital y optimizando así el funcionamiento del microcontrolador.

Por otro lado, el archivo *“setup.ino”* se encarga de la inicialización del hardware del sistema. En este módulo se configuran todos los pines del ESP32 como entradas o salidas según su función dentro del proyecto. Los LEDs asociados al estado de funcionamiento y al llenado de los palets se configuran como salidas digitales, mientras que los botones físicos se inicializan como entradas con resistencias internas *pull-up*, permitiendo detectar correctamente las pulsaciones. Asimismo, durante la fase de configuración inicial se establece el estado por defecto de los LEDs externos y se asegura que el LED interno del dispositivo permanezca apagado al arrancar el sistema.

En tercer lugar, se encuentran los archivos encargados de gestionar el manejo de señales que determinará el funcionamiento de las simulaciones explicadas anteriormente. Estos módulos permiten establecer la comunicación entre las acciones realizadas físicamente mediante los pulsadores y la respuesta visual del sistema a través de los LEDs de estado.

El archivo *“comunicacion_buttons.ino”* implementa toda la lógica relacionada con la lectura y gestión de los botones físicos conectados al ESP32. Para ello, se utiliza la estructura *BotonMQTT*, definida en el archivo auxiliar *“buttons.h”*, donde se almacenan los parámetros necesarios de cada botón, como el pin asociado, el *topic* MQTT de publicación y el mensaje que debe enviarse. Gracias a esta estructura, es posible gestionar múltiples botones de manera más organizada y escalable.

Dentro de este módulo destaca la función *gestionarBoton()*, encargada de aplicar un sistema de *debounce* por software para evitar falsas detecciones provocadas por el rebote mecánico de los pulsadores. Cuando se detecta una pulsación válida, el sistema publica automáticamente un mensaje MQTT asociado al botón presionado. De esta manera, cada pulsador puede generar distintos eventos dentro de la simulación, como la creación de azulejos buenos, malos o defectuosos, así como el vaciado de los diferentes palets. Asimismo, el archivo incorpora la función *on_loop()*, ejecutada continuamente durante el funcionamiento del programa. Esta función recorre todos los botones definidos y actualiza constantemente su estado, permitiendo detectar pulsaciones en tiempo real y garantizar una comunicación fluida entre el hardware y la simulación.

Por otro lado, el archivo *“comunicacion_leds.ino”* se encarga de gestionar la recepción de mensajes MQTT y el control visual del sistema mediante LEDs. En primer lugar, la función *suscribirseATopics()* realiza la suscripción a los distintos *topics* relacionados con el estado de funcionamiento y con el llenado de los palets. Posteriormente, la función *alRecibirMensajePorTopic()* procesa los mensajes recibidos y actúa en consecuencia sobre los LEDs correspondientes. De este modo, cuando se recibe un mensaje indicando el estado de un palet, el sistema enciende o apaga el LED asociado para reflejar visualmente si dicho palet se encuentra lleno o vacío. De manera similar, el LED de funcionamiento permite indicar el estado operativo general de la simulación. Además, el sistema incorpora mensajes

de depuración mediante consola serie, facilitando el seguimiento de los eventos recibidos y la detección de posibles errores o *topics* no reconocidos.

Por último, se encuentra el archivo principal “ESP32-S3.ino”, encargado de coordinar y ejecutar de manera conjunta todos los módulos explicados anteriormente. Este archivo actúa como núcleo principal del programa, integrando las funciones de configuración, comunicación y control necesarias para el correcto funcionamiento del sistema basado en ESP32-S3 y MQTT.

El archivo incluye todas las librerías necesarias para la gestión de la conectividad WiFi y la comunicación MQTT. Además, se define un identificador único para el dispositivo utilizando la dirección MAC del ESP32, permitiendo diferenciar cada dispositivo conectado dentro de la red MQTT.

La función *setup()* realiza toda la inicialización principal del sistema. Durante esta fase, se configuran las comunicaciones serie para tareas de depuración, se establece la conexión a la red WiFi y posteriormente se realiza la conexión con el broker MQTT utilizando el identificador del dispositivo como *client-id*. Una vez establecida la comunicación, el sistema ejecuta las funciones encargadas de suscribirse a los *topics* necesarios y de configurar el hardware definido en los módulos auxiliares.

Por otro lado, la función *loop()* constituye el ciclo principal de ejecución continua del programa. En cada iteración se supervisa el estado de la conexión WiFi, se mantiene activa la comunicación MQTT y se ejecutan las funciones encargadas de gestionar botones, LEDs y eventos asociados a la simulación. Gracias a esta estructura modular, el sistema consigue mantener una comunicación constante y sincronizada entre el entorno físico y el entorno virtual de simulación.

En conjunto, este archivo principal permite integrar todos los componentes desarrollados previamente, asegurando el correcto intercambio de información entre sensores, actuadores y la plataforma MQTT, así como la coordinación general del sistema embebido implementado.

En relación con la arquitectura implementada en la API de Python para RoboDK, se fundamenta en la extensión de las capacidades del RoboDK mediante el uso exclusivo de su API de Python. Esta integración permite transformar un entorno de simulación estático en un sistema de control conectado. Gracias al broker MQTT y el sistema embebido se hace posible una comunicación y coordinación efectiva entre los distintos elementos que participan en el proceso. Para ello se ha requerido de la implementación de diferentes scripts de Python. Cabe destacar que se ha llevado a cabo una distinción en los scripts; en primer lugar se encuentran los scripts cuya función es el envío de mensajes a través de MQTT con el propósito de cambiar los indicadores LEDs, y en la segunda parte de la distinción se encuentran aquellos scripts encargados de recibir los mensajes para la ejecución de los diferentes programas de manera coordinada.

En relación con los scripts en Python cuya finalidad es la lectura de los mensajes recibidos, éstos se han diseñado bajo una jerarquía de *topics* segmentada de la siguiente manera; por un lado se encuentran los scripts encargados de la suscripción al *topic* adecuado y los scripts que llevan a cabo la ejecución de la tarea correspondiente que se ha indicado del primer script, comentado anteriormente.

Los scripts denominados “*LeerMQTTS*” y “*LeerMQTTB*” son los encargados de suscribirse al *topic* requerido. Estos ficheros están constantemente realizando la lectura del broker MQTT hasta recibir un mensaje. Una vez recibido el mensaje, éste se guarda y se envía a la segunda parte del proceso.

La segunda parte del proceso consiste en que los scripts “*RobotControllerS*” y “*RobotControllerB*” reciben el mensaje que los scripts anteriormente mencionados han enviado y realizan su función principal, la cual consiste en la ejecución de las tareas correspondientes que se han indicado a través del broker MQTT. Sobre el script “*RobotControllerS*” cabe mencionar que es el encargado de spawnear los azulejos. Para ello, una vez se ha recibido el dato, éste se traslada a la variable global denominada TipoAzulejo. Esta variable es la responsable de determinar el estado del próximo azulejo que se va a empaquetar en la caja correspondiente a su estado. Sin embargo, por lo que se refiere al script “*RobotControllerB*” es quien administra las tareas de tipo de azulejo, retirada de palets llenos o apagado del LED. Esto es posible ya que en base al botón que se haya presionado en la celda de trabajo, se llevará a cabo una de esas tareas.

Por otro lado, en materia de aquellos programas encargados de enviar mensajes al broker MQTT, de manera similar a los script de lectura, también están diseñados bajo una jerarquía en base a la eventualidad que activa el encendido del indicador visual -LEDs- ; por un lado se encuentra el script que avisa de la existencia de un proceso activo y, seguidamente se ubican los scripts cuyo aviso se activa cuando uno o ambos palets se encuentran en su máxima capacidad. Cada programa detallado anteriormente cuenta con dos scripts; uno es el encargado de enviar al broker el mensaje “ON”, mientras que el otro, de manera equivalente envía el mensaje “OFF”. En el caso de estos scripts, es de vital relevancia mencionar que a diferencia de los script de lectura, los cuales estaban en constante funcionamiento, éstos solamente se ejecutan una sola repetición, ya que la estructura definida incluye la suscripción al *topic* deseado para la eventualidad y el mensaje pertinente.

Como instancia final, la celda de trabajo implementada en RoboDK se encuentra vinculada a la Base de Datos centralizada mediante el controlador en Python, lo que permite una integración de elementos como los azulejos o las cajas -tanto vacías como llenas- en la capa de ejecución de la simulación de la celda de trabajo. Esta conexión garantiza la persistencia y trazabilidad de cada ciclo productivo, registrando de forma automática variables como el estado de calidad de los azulejos, avisos y eventos de paletizado. Al consolidar esos datos en un sistema de gestión, se facilita la monitorización del rendimiento

de la planta, asegurando que cada acción realizada en la simulación esté registrado de manera permanente para un posterior control.

En conclusión, esta infraestructura de datos se integra de manera sinérgica con el sistema integrado de comunicaciones. El conjunto de scripts en Python permite una comunicación bidireccional y sin interrupciones con el protocolo MQTT y la ESP32-S3. Gracias a este sistema de envío y recepción de datos en tiempo real, se logra que el proceso de automatización funcione de manera fluida, sincronizada y totalmente conectada, garantizando una buena coordinación entre el entorno virtual y el hardware físico.

5. Integración con otras asignaturas

Respecto a la asignatura de Programación Avanzada (PRA), se ha evaluado la posibilidad de integrar algoritmos de datos complejos como árboles, tablas *hash* o algoritmos de tipo *Divide y Vencerás*. Finalmente, en el firmware del ESP32-S3 se ha decidido integrar un mecanismo de deduplicación de mensajes MQTT con el fin de evitar el reproceso de mensajes idénticos recibidos en una ventana corta de tiempo. La necesidad de esta funcionalidad surge de un problema habitual en entornos de automatización industrial, donde las retransmisiones del broker, las reconexiones de red o los reenvíos accidentales pueden provocar la recepción repetida de un mismo comando o evento, con el riesgo de generar activaciones duplicadas, inconsistencias en el estado del sistema y un consumo innecesario de recursos. Dado que la plataforma objetivo es un microcontrolador con memoria y capacidad de cálculo limitadas, se ha optado por una solución de bajo coste computacional, de integración sencilla y fácilmente verificable en condiciones reales de operación.

La solución implementada consiste en calcular un hash no criptográfico sobre la concatenación del topic y el payload recibidos, y comparar dicho valor con un buffer circular que almacena los N hashes más recientes. Si el hash ya figura en la estructura, el mensaje se considera duplicado y se descarta; en caso contrario, se procesa normalmente y su hash sustituye a la entrada más antigua del buffer. Esta lógica se incorpora directamente en la rutina de callback de MQTT del firmware, de forma que la comprobación se realiza en el punto de entrada de los mensajes, reduciendo así el impacto sobre el resto de la arquitectura. Desde el punto de vista algorítmico, la solución ocupa una memoria fija de $O(N)$ y presenta un coste de comprobación proporcional al tamaño del buffer, lo que resulta asumible para valores reducidos de N, como los empleados en el prototipo, y compatible con las tasas de mensajes previstas en la planta.

La elección de este enfoque responde, entre otros, a mantener una huella de memoria pequeña y garantizar una latencia de respuesta adecuada para el control en tiempo real. Además, se han considerado aspectos de robustez, como el riesgo de colisiones inherente al uso de hashes y el hecho de que la ventana de deduplicación se define por número de mensajes y no por tiempo. Aun así, para el escenario actual la solución ofrece una mejora

tangible en fiabilidad, al reducir reprocesos, falsos disparos y operaciones redundantes como logs, escrituras en base de datos o envíos innecesarios a actuadores. En consecuencia, la deduplicación por hash y buffer circular constituye una medida práctica, justificada y eficaz para el contexto del ESP32-S3, con potencial de ampliación futura si las necesidades operativas lo requieran.

Por otro lado, con respecto al desarrollo y justificación de estructuras de datos avanzados se ha empleado una estructura de datos compuesta con la finalidad de organizar de manera eficiente la gestión de los botones físicos y su relación con los eventos MQTT. Esta estructura se puede observar concretamente en *buttons.h*. Este fichero define la estructura *ButtonMQTT*, cuya finalidad es agrupar en una única unidad lógica todos los atributos necesarios para cada botón: el pin de entrada, el topic MQTT asociado, el payload a publicar y variables internas de estado utilizadas en el control del rebote. Este método de organización permite encapsular la información relevante de cada elemento y mejora la legibilidad, mantenibilidad y coherencia del código.

De modo similar, en *05_comunicacion_buttons.ino* se utiliza un array estático de estructuras *ButtonMQTT* para el procesamiento de varios botones de manera uniforme mediante un único bucle. Gracias a la organización implementada, la lógica de lectura, antirrebote y publicación MQTT se aplica de forma homogénea a todos los dispositivos de entrada. De esta forma, se evita duplicar el código y facilita la escalabilidad del sistema. Si en el futuro se incorporan nuevos botones, solamente se requerirá añadir un nuevo elemento al array, sin necesidad de modificar el resto del código. Por tanto, aunque no se recurre a técnicas algorítmicas complejas, sí se aplica una estructura de datos adecuada y justificada, basada en la encapsulación de información y en el tratamiento colectivo de múltiples elementos.

Respecto a la asignatura de Gestión de Datos e Información (GDI), se ha diseñado e implementado el sistema de información detallado a continuación en este punto (5). Este desarrollo abarca desde la definición de los requisitos de datos hasta la creación de los scripts de persistencia en Python. A través de esta integración, se asegura que toda la actividad de la planta (estados de azulejos, movimientos de los robots y registros de producción) quede almacenada correctamente en una base de datos relacional, permitiendo así el análisis de los KPIs y la trazabilidad de todo el proceso de empaquetado. En cuanto a los scripts utilizados para la creación de la Base de Datos están adjuntos también en el repositorio de Github, situados en la carpeta de *Base-Datos* donde se ha procedido a una breve explicación del funcionamiento de la misma.

5.1. Descripción del Sistema de Información

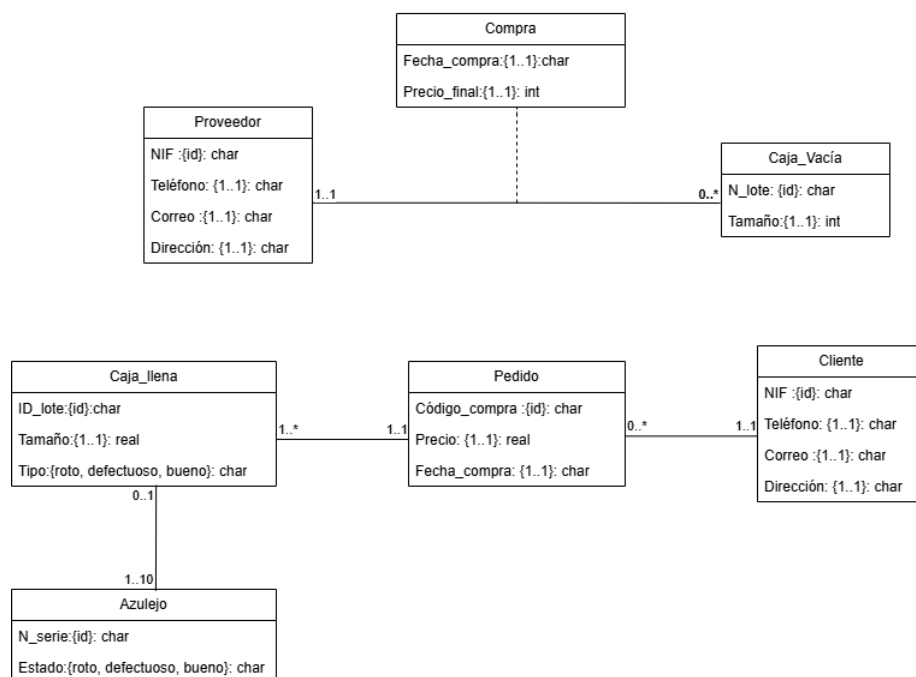
En una fábrica de azulejos se necesita un sistema de gestión de datos. Los azulejos a procesar son identificados por su número de serie y su estado. En cuanto al inventario utilizado en el proceso, se necesita saber información acerca de las cajas de embalaje. Durante el proceso, hay dos tipos de cajas definidas por identificador del lote y por su tamaño

o capacidad. Además, las cajas llenas cuentan con una etiqueta que indica el estado de los azulejos que guarda.

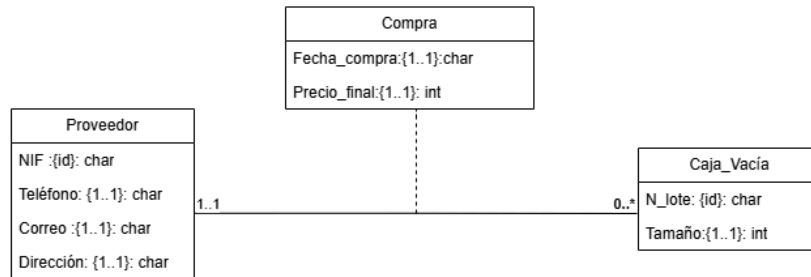
Por otra parte, cada cliente se corresponde con una empresa con nombre, identificador, teléfono de contacto, correo y dirección. Asimismo, también es necesario conocer los proveedores de cajas mediante el NIF de la empresa, el número de contacto y correo. De este modo, por cada compra de cajas vacías, se archivará la fecha de la compra y el precio final, ya que hay distintas opciones de compra. Por último, por cada pedido de cajas llenas de un cliente que se tramite, se guardará el código de compra como identificador, el precio y la fecha de la compra.

5.2. Diagrama UML

Diagrama UML derivado de la descripción anterior.



El modelo conceptual representado en base a la descripción del sistema de información ha sido diseñado con el fin de representar el proceso desde la adquisición de suministros, como las cajas, hasta la entrega final al cliente. Como se puede observar, la estructura utilizada presenta una clara distinción entre entidades externas que proveen de materiales requeridos para el proceso de automatización y procesos que intercambian diferentes informaciones, como el llenado de las cajas de diferente tipo de azulejo.

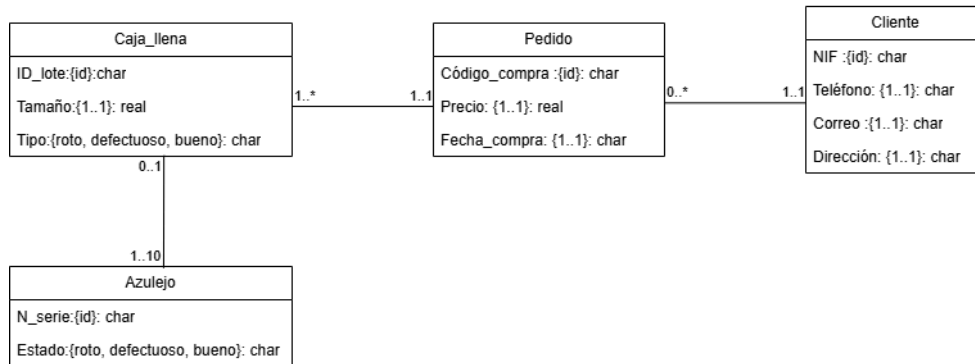


En primer lugar, se va a proceder a comentar el bloque de entidades externas.

Primeramente, se define la clase Proveedor, cuyo identificador es el NIF, con atributos de obligatoriedad ($\{1..1\}$) que permiten asegurar en todo momento la localización de la empresa que provee de cajas a la fábrica de azulejos. Dicho esto implica la existencia de una nueva clase, Caja_Vacia. Cada caja contiene su identificador a través del número de lote y un único tamaño ($\{1..1\}$), ya que las cajas solamente se van a utilizar con el fin de ser llenadas por azulejos de un único tamaño de igual manera, por lo que no se precisa de diferentes modelos de tamaño de cajas.

De este modo, ambas clases requieren de una relación que las asocie. Sin embargo, esta relación no se trata de una relación simple, sino que se gestiona a través de la clase asociativa Compra, que cada vez que se realiza una compra se almacena la fecha y el precio final al que asciende la compra de cajas al proveedor correspondiente. En consecuencia, los atributos Fecha_compra y Precio_final se almacenan como datos propios de la acción de compra y no del objeto, permitiendo que un mismo tipo de caja sea adquirida en distintos momentos y a precios variables.

Para finalizar, cabe destacar que la cardinalidad 1..1 en el lado de la clase Proveedor en la relación de Compra garantiza que cada lote de cajas vacías tenga siempre un responsable de suministro de cajas identificado. Mientras que en el otro extremo del Proveedor (0..*) indica que un proveedor puede ser registrado en el sistema de la Base de Datos antes de haberse realizado la primera compra.



Para finalizar con el diagrama UML a continuación se va a justificar la segunda parte del esquema conceptual.

Antes que nada, cabe recalcar que esta segunda parte corresponde al núcleo de producción de la fábrica. Así bien, se pueden observar diferentes clases; Caja_Llena, Azulejo, Pedido y Cliente.

En este caso, la clase con más importancia es la de Azulejo, ya que depende de qué tipo de azulejo se trate tendrá un valor u otro.

La clase Azulejo consta de su identificador denominado número de serie, al igual que en el caso de las cajas. Este identificador es único, por lo que cada uno de los azulejos que se vayan a paletizar contarán con uno propio. Asimismo, al tratarse de un proceso que clasifica por tipos de azulejos para su posterior paletización y empaquetado, obliga a la creación de un atributo denominado Estado. Este atributo está restringido solamente a tres tipos de estado: roto, defectuoso o bueno. Esta clasificación lo que permite al sistema es la automatización de los filtros de calidad que se llevan a cabo en la fábrica.

La clase Azulejos está directamente relacionada a la clase Caja_Llena, puesto que cada azulejo se depositará en el interior de una caja hasta llenarla. En cuanto a las cardinalidades de la relación, se puede observar que se ha establecido una cardinalidad de 1..10 en el extremo del Azulejo. Dicha cardinalidad lo que trata de establecer es que una caja llena debe contener obligatoriamente al menos un azulejo y, por temas de capacidad de las cajas, no puede exceder los 10 azulejos en su interior. En cambio, por el extremo de Caja_Llena se encuentra la cardinalidad 0..1, lo cual permite que un azulejo exista en el inventario de la fábrica de forma individual pero no tiene por qué estar asignado a un número de lote de una caja.

En cuanto a la clase Caja_Llena cuenta con los mismos atributos que la clase anteriormente comentada, Caja_Vacía, pero con una ligera modificación. Cada caja llena se identifica con su número de lote; además cuenta con un atributo obligatorio ({1..1}) denominado tamaño y, un atributo denominado Tipo. Este atributo hace la misma función que

el atributo Estado de la clase Azulejo, almacena qué tipo de azulejo contiene en su interior, puesto que dependiendo del tipo de azulejo del que se trate la línea de paletizado y empaquetado es diferente.

Siguiendo el diagrama, la siguiente clase que aparece es la clase Pedido. En este caso, el atributo que actúa como identificador y por lo tanto es único es el código de compra. También, cuenta con los atributos de precio y fecha de compra con restricciones de obligatoriedad.

Previo a continuar con el resto de clases, es de vital importancia explicar las asociaciones que existen entre la clase Caja_Llena y Pedido, y, Cliente y Pedido, ya que se trata de la fase final del proceso que relaciona la producción con el mercado.

En torno a la asociación entre las clases Caja_Llena y Pedido se observa la cardinalidad 1..1 desde la Caja_Llena hacia el Pedido. Esta cardinalidad lo que indica es la garantía que de una caja específica, una vez se ha vendido, queda bloqueada y vinculada a ese código de compra, por lo que no se puede dar la circunstancia de que se lleve a cabo una doble venta del mismo producto. En cambio, en el otro extremo de la relación cuenta con una cardinalidad 1..* lo cual obliga a que cualquier pedido que se haya registrado contenga como mínimo, una caja llena. Esta cardinalidad lo que busca es evitar la creación de pedidos de venta vacíos, puesto que carece de sentido.

Seguidamente, en el caso de la relación entre la clase Pedido y Cliente se observan las cardinalidades 0..* a 1..1 lo que indica que la base de datos de la empresa de la fábrica de azulejos contiene información de posibles clientes futuros aunque aún no hayan tramitado ninguna compra. De igual manera, esta asociación asegura que cada pedido que se realiza está vinculado a un único cliente.

Finalmente, en último lugar se encuentra la clase Cliente. Esta clase almacena toda la información de aquellas empresas o individuos que realizan un pedido a la fábrica de azulejos. Para lograr un buen funcionamiento en esta clase se almacenará el NIF de aquella empresa que tramita el pedido, lo cual servirá como identificador, además de un teléfono de contacto, una dirección y un correo. Todos los atributos de esta clase son obligatorios, de manera que al igual que en el anterior caso se ha comentado que en la clase Proveedores se exigen todos estos atributos con la finalidad de asegurar en todo momento la localización de la empresa que provee de cajas a la fábrica de azulejos. En este caso, se busca la misma finalidad desde la fábrica de azulejos hacia el cliente.

5.3. Esquema Lógico - Relacional

El resultado de la traducción del diagrama UML al modelo relacional para las relaciones externas y de producción es el siguiente :

Proveedor(NIF: char, Teléfono: char, Correo: char, Dirección: char, N_Lote: char)

CP: {NIF}

VNN: {Teléfono, Correo, Dirección}

CAj: {NIF} -> Caja_Vacia(N_Lote)

Caja_Vacia(N_Lote: char, Tamaño: int)

CP: {N_Lote}

VNN: {Tamaño}

Compra(Proveedor: char, Caja_Vacia: char, Fecha_Compra: char, Precio_Final: int)

CP: {Caja_Vacia}

VNN: {Proveedor, Fecha_Compra, Precio_Final}

CAj: {Proveedor} -> Caja_Vacia(N_Lote)

CAj: {Caja_Vacia} -> Proveedor(NIF)

Caja_Llena(ID_lote: char, tamaño: real, Tipo: char, Código_compra: char)

CP: {ID_lote}

VNN: {tamaño, Código_compra}

Check: Tipo ∈ {'roto', 'defectuoso', 'bueno'}

CAj: {Código_compra} --> Pedido(Código_compra)

R11: Todo valor de Código_compra en Pedido debe aparecer en Código de compra de Cajas llenas.

Azulejo(N_serie: char, Estado: char, ID_lote: char)

CP: {N_serie}

CAj: {ID_lote} --> Cajas llenas (ID_lote)

Check: Estado ∈ {'roto', 'defectuoso', 'bueno'}

R11: Todo valor de ID_lote en Cajas llenas debe aparecer en ID_lote de Azulejos.

Pedido(Código_compra: char, Precio: real, Fecha_compra: char, NIF: char)

CP: {Código_compra}

VNN: {Precio, Fecha_compra, NIF}

CAj: {NIF} --> Cliente (NIF)

Cliente(NIF: char, Teléfono: char, Correo: char, Dirección: char)

CP: {NIF}

VNN: {Teléfono, Correo, Dirección}

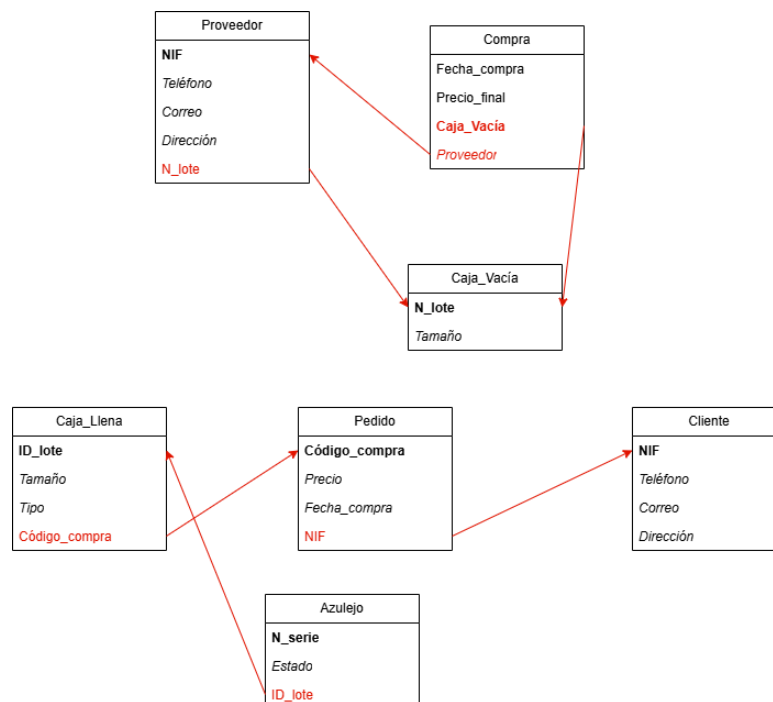
Así pues, respecto a las restricciones de integridad del esquema conceptual que no se han podido implementar al esquema relacional a través de tablas y claves han sido las siguientes:

1. Restricción de multiplicidad numérica (1..10): El modelo relacional permite asegurar que un Azulejo pertenece a una Caja_Llena a través de una clave ajena. Sin embargo, no se puede limitar que una caja contenga máximo 10 registros de azulejos.
2. Participación total en Pedidos (1..*): En el modelo relacional, primero se crea el Pedido y luego se le asigna una Caja, por lo que esto implica que en un

primer instante el sistema permite crear al cliente un pedido vacío aunque sea de manera temporal.

3. Restricciones de Estado/Tipo: En el modelo relacional no se puede garantizar que una Caja_Llena que contenga Azulejos de Tipo “bueno” albergue Azulejos todos ellos en un Estado “bueno”. En síntesis, la lógica de la fábrica se basa en que ambas etiquetas (Tipo/Estado) deben coincidir con el Estado de los azulejos. No obstante, no se puede garantizar su verificación.

Para finalizar, con respecto al esquema gráfico sobre el modelo relacional de la Base de Datos se va a representar de la siguiente manera:



Lo primero a destacar es que cada clave primaria de una relación se representa en **negrita**. Por otra parte, los atributos con restricciones de valor no nulo (VNN) se pueden identificar por estar en formato *cursiva*, como se trata de los atributos de *Tamaño*, *Teléfono*, *Correo*, etc. En cambio, si se trata de un atributo con restricción de unicidad (Uni) en este caso se identifica por el subrayado con el que cuenta. En el caso de la base de datos de la fábrica de azulejos no cuenta con ningún atributo con esta restricción. Finalmente, para la representación de las claves ajenas, éstas se representan en color **rojo** y además, cuenta con una flecha apuntando al atributo referenciado como se puede observar en el esquema anterior.

6. Calidad y Adecuación del Proyecto

La calidad y adecuación de este proyecto se fundamentan en la implementación de una arquitectura de integración robusta que conecta el nivel de campo con la gestión de datos industriales. La solución técnica destaca por el uso del protocolo MQTT y el formato JSON, lo que permite un desacoplamiento efectivo entre los dispositivos físicos, como el ESP32, y el entorno de simulación en RoboDK. Esta elección tecnológica garantiza la interoperabilidad y escalabilidad del sistema, permitiendo que la celda de fabricación se adapte a futuros sensores o cambios en la línea de producción sin comprometer la estructura de comunicaciones.

Para asegurar la fiabilidad en la captación de datos, se ha desarrollado un software para el ESP32-S3 basado en una estructura de programación modular compuesta por nueve archivos especializados. Esta organización del código no solo facilita la mantenibilidad, sino que garantiza la estabilidad del sistema mediante módulos críticos como *wifi_lib.ino* y *mqtt_lib.ino* para la conectividad persistente, y *logger.ino* para la depuración avanzada por niveles de severidad. La adecuación técnica se ve reforzada por la implementación de sistemas de debounce por software en la gestión de pulsadores físicos y una lógica de control de LEDs sincronizada mediante callbacks de MQTT, asegurando que cualquier interacción física se traduzca instantáneamente en un evento dentro de la simulación virtual.

La implementación de esta arquitectura basada en Python y MQTT ha permitido establecer una comunicación bidireccional y en tiempo real entre RoboDK y el sistema embebido. La distinción entre scripts de control y de monitorización asegura una ejecución precisa de las tareas de paletizado y la gestión de indicadores visuales. Además, la vinculación con una base de datos centralizada eleva el sistema de una simple simulación a una herramienta de gestión industrial, permitiendo el registro automático de eventos y la trazabilidad de la calidad, lo que asegura una sincronización total entre el gemelo digital y los componentes físicos.

Además, la adecuación del proyecto se refleja en el cumplimiento estricto de los KPIs definidos, especialmente en la optimización del tiempo de ciclo mediante scripts de Python y la mejora en el control de calidad, que permite clasificar automáticamente los azulejos y reducir drásticamente las mermas. Desde el punto de vista del software, la solución es altamente mantenible gracias a una estructura modular que separa la lógica de movimiento del robot de la gestión de mensajes, facilitando actualizaciones futuras. Finalmente, la viabilidad económica, con un retorno de inversión (ROI) optimizado, y la mejora en las condiciones ergonómicas de la planta confirman que el diseño no es solo una simulación teórica, sino una respuesta técnica profesional y adecuada a las exigencias reales de la automatización industrial moderna.